Exercise 1: Basic Vector Operations

```
import numpy as np

# Create two vectors
v1 = np.array([1, 2, 3])
v2 = np.array([4, 5, 6])

# 1. Vector addition
def vector_addition(a, b):
    return np.add(a, b)

# 2. Vector subtraction
def vector_subtraction(a, b):
    return np.subtract(a, b)

# 3. Dot product
def dot_product(a, b):
    return np.dot(a, b)

# 4. Cross product (for 3D vectors)
def cross_product(a, b):
    return np.cross(a, b)

# 5. Vector magnitude
def vector_magnitude(a):
    return np.linalg.norm(a)

# Test your functions
print("Addition:", vector_addition(v1, v2))
print("Subtraction:", vector_subtraction(v1, v2))
print("Dot product:", dot_product(v1, v2))
print("Cross product:", cross_product(v1, v2))
print("Magnitude v1:", vector_magnitude(v1))
print("Magnitude v2:", vector_magnitude(v2))
```

```
Addition: [5 7 9]
Subtraction: [-3 -3 -3]
Dot product: 32
Cross product: [-3  6 -3]
Magnitude v1: 3.7416573867739413
Magnitude v2: 8.774964387392123
```

Exercise 2: Basic Matrix Operations

```
# Create two matrices
m1 = np.array([[1, 2], [3, 4]])
m2 = np.array([[5, 6], [7, 8]])

# 1. Matrix addition
def matrix_addition(a, b):
    return np.add(a, b)
```

```
# 2. Matrix subtraction
def matrix_subtraction(a, b):
    return np.subtract(a, b)

# 3. Matrix multiplication
def matrix_multiplication(a, b):
    return np.matmul(a, b)

# 4. Matrix transpose
def matrix_transpose(a):
    return np.transpose(a)

# 5. Matrix determinant (for square matrices)
def matrix_determinant(a):
    return np.linalg.det(a)

# Test your functions
print("Matrix addition:\n", matrix_addition(m1, m2))
print("Matrix subtraction:\n", matrix_subtraction(m1, m2))
print("Matrix multiplication:\n", matrix_multiplication(m1, m2))
print("Matrix transpose m1:\n", matrix_transpose(m1))
print("Matrix determinant m1:", matrix_determinant(m1))
```

```
Matrix addition:
 [[ 6  8]
 [10 12]]
Matrix subtraction:
 [[-4 -4]
 [-4 -4]]
Matrix multiplication:
 [[19 22]
 [43 50]]
Matrix transpose m1:
 [[1 3]
 [2 4]]
Matrix determinant m1: -2.0000000000000004
```

Exercise 3: Vector Transformations and Angle Calculation

```
# Sample vectors
v1 = np.array([1, 2, 3])
v2 = np.array([4, 5, 6])
v = np.array([1, 0])

# 1. Vector scaling
def scale_vector(v, scalar):
    return scalar * v

# 2. Angle between two vectors (in radians)
def angle_between_vectors(a, b):
    cos_theta = np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
    return np.arccos(cos_theta)

# 3. Vector projection of a onto b
def vector_projection(a, b):
    return (np.dot(a, b) / np.dot(b, b)) * b
```

```
# 4. Rotate a 2D vector by theta radians
def rotate_vector_2d(v, theta):
    rotation_matrix = np.array([
        [np.cos(theta), -np.sin(theta)],
        [np.sin(theta),  np.cos(theta)]
    ])
    return np.dot(rotation_matrix, v)

# Test your functions
print("Scaled vector:", scale_vector(v, 2.5))
print("Angle between v1 and v2:", angle_between_vectors(v1, v2))
print("Projection of v1 onto v2:", vector_projection(v1, v2))
print("Rotated vector:", rotate_vector_2d(v, np.pi/2))  # 90 degrees
```

```
Scaled vector: [2.5 0. ]
Angle between v1 and v2: 0.2257261285527342
Projection of v1 onto v2: [1.66233766 2.07792208 2.49350649]
Rotated vector: [6.123234e-17 1.000000e+00]
```

Exercise 4: Eigenvalues and Eigenvectors

```
# 1. Compute eigenvalues and eigenvectors
def compute_eigen(matrix):
    eigenvalues, eigenvectors = np.linalg.eig(matrix)
    return eigenvalues, eigenvectors

# 2. Verify that Av = lambda * v for the first eigenvalue/eigenvector pair
def verify_eigen(matrix, eigenvalues, eigenvectors):
    # First pair selection
    lambda_1 = eigenvalues[0]
    v1 = eigenvectors[:, 0]  # NumPy returns eigenvectors as column vectors

    # Calculate LHS (A * v) and RHS (lambda * v)
    lhs = np.dot(matrix, v1)
    rhs = lambda_1 * v1

    print(f"\nVerification for first eigenpair:")
    print("A @ v1  :", lhs)
    print("λ1 * v1 :", rhs)
    print("Is Av ≈ λv?", np.allclose(lhs, rhs))

# Test with a sample matrix
sample_matrix = np.array([[2, -1], [-1, 2]])
eigenvalues, eigenvectors = compute_eigen(sample_matrix)

print("Eigenvalues:", eigenvalues)
print("Eigenvectors:\n", eigenvectors)

verify_eigen(sample_matrix, eigenvalues, eigenvectors)
```

```
Eigenvalues: [3. 1.]
Eigenvectors:
 [[ 0.70710678  0.70710678]
 [-0.70710678  0.70710678]]
```

```
Verification for first eigenpair:
A @ v1  : [ 2.12132034 -2.12132034]
λ1 * v1 : [ 2.12132034 -2.12132034]
Is Av ≈ λv? True
```

Exercise 5: Grouping Data Points by Angular Similarity

```
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt

# Load the Iris dataset
iris = load_iris()
X = iris.data[:, :2]  # Using only sepal length and width
y = iris.target

def calculate_angle(a, b):
    """Calculate the angle between two vectors in degrees"""
    # 1. Normalize vectors a and b (convert to unit vectors)
    a_norm = a / np.linalg.norm(a)
    b_norm = b / np.linalg.norm(b)

    # 2. Compute dot product (clip for floating point precision issues)
    cos_theta = np.clip(np.dot(a_norm, b_norm), -1.0, 1.0)

    # 3. Use arccos to get angle in radians
    angle_rad = np.arccos(cos_theta)

    # 4. Convert to degrees
    angle_deg = np.degrees(angle_rad)

    return angle_deg

def group_by_angle(data, angle_threshold=30):
    """Group data points based on angular similarity"""
    n_samples = data.shape[0]
    groups = np.zeros(n_samples, dtype=int)
    current_group = 1  # 1-indexed keeping 0 as unassigned check

    for i in range(n_samples):
        if groups[i] == 0:  # If point isn't assigned yet
            groups[i] = current_group
            for j in range(i + 1, n_samples):
                if groups[j] == 0:
                    angle = calculate_angle(data[i], data[j])
                    # If angle < threshold, assign to same group
                    if angle < angle_threshold:
                        groups[j] = current_group
            current_group += 1
    return groups

# Run grouping
angle_threshold = 30  # degrees
groups = group_by_angle(X, angle_threshold)

# Visualize results
```

```
# plt.figure(figsize=(10, 6))
# plt.scatter(X[:, 0], X[:, 1], c=groups, cmap='tab20', alpha=0.7)
# plt.title(f"Data Grouping by Angle (Threshold: {angle_threshold}°)")
# plt.xlabel("Sepal Length (cm)")
# plt.ylabel("Sepal Width (cm)")
# plt.colorbar(label="Group ID")
# plt.grid(True)
# plt.show()

# 1. Background Setting: Poori figure ko black kar dein
plt.figure(figsize=(10, 6), facecolor='black') # figure outside color
ax = plt.gca()
ax.set_facecolor('black') # plot area color

# 2. Main Scatter Plot: Green scheme ke saath
scatter = plt.scatter(
    X[:, 0],
    X[:, 1],
    c=groups,
    # 'lime' vibrant hai, 'Greens' gradient ke liye use kar sakty hain
    cmap='Greens',          # Vibrant, bright green look (agar smooth gradient chahye to 'Greens_r' use karein)
    alpha=1.0,            # Black par transparency low rakhein taake glow zyada aye
    s=70,                 # Thora larger size
    edgecolors='white',   # Subtle glow effect ke liye white border
    linewidth=0.5
)

# 3. Text and Labels customization (Must be light/white)
plt.title(f"Data Grouping by Angle (Threshold: {angle_threshold}°)", fontsize=13, color='#CCFFCC', fontweight='medium', pad=12) # Pale green title
plt.xlabel("Sepal Length (cm)", fontsize=10, labelpad=8, color='white')
plt.ylabel("Sepal Width (cm)", fontsize=10, labelpad=8, color='white')

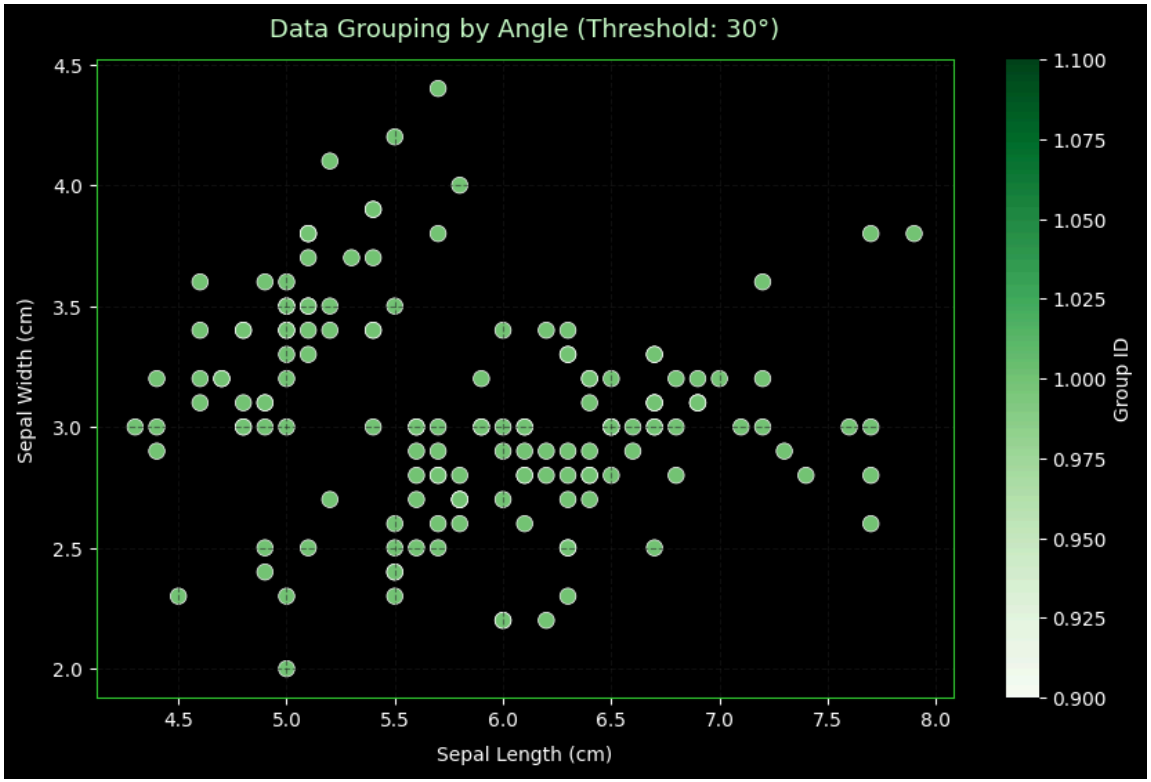
# 4. Axes, Ticks, and Grid colors
# Ticks labels color change karein
ax.tick_params(axis='x', colors='white')
ax.tick_params(axis='y', colors='white')

# Spines (borders) ko white ya pale green kar dein
for spine in ax.spines.values():
    spine.set_color('#33CC33') # Pale green outline

# 5. Colorbar style change (Black theme suitable)
cbar = plt.colorbar(scatter)
cbar.set_label("Group ID", color='white')
cbar.ax.yaxis.set_tick_params(color='white') # ticks on colorbar
plt.setp(plt.getp(cbar.ax.axes, 'yticklabels'), color='white') # labels on colorbar
cbar.outline.set_visible(False) # borders around colorbar remove karein

# 6. Subtle dark grid style
plt.grid(True, linestyle='--', color='#222222', alpha=0.5) # Bohot light grey grid

plt.show()
```


Data Grouping by Angle (Threshold: 30°)
Sepal Width (cm)
Sepal Length (cm)
Group ID
4.5
4.0
3.5
3.0
2.5
2.0
4.5
5.0
5.5
6.0
6.5
7.0
7.5
8.0
1.100
1.075
1.050
1.025
1.000
0.975
0.950
0.925
0.900